

Space Salvager

Final Project - AI-Based Game Development

Python | Pygame | OpenCV | MediaPipe | DeepFace | Gymnasium | Stable-Baselines3

Submitter: [your name + ID] **Repository:** github.com/LTJohnShepherd/space-salvager

1. Game Description

Space Salvager is a 2D arcade game built with Pygame. The player pilots a single salvager ship in an asteroid field. Flying into an ore pickup scores a point and spawns new ore; touching any of the roaming pirate cruisers destroys the ship and ends the run. The goal is to collect a target number of ore (15) without colliding with a pirate.

Rules & ending. Each ore = +1 score. A pirate collision ends the episode (failure). The episode also ends on reaching the target score (win) or exceeding the step limit. The same game object is driven three ways: keyboard (manual), webcam hand gesture + facial emotion (camera), and a trained reinforcement-learning agent (AI).

2. Role of Each Library

- **Pygame** - runs the game loop, input, rendering and sound (game/space_game.py, manual_play.py).
- **OpenCV** - opens the webcam (VideoCapture), reads and flips frames, converts BGR to RGB, draws overlays, and shows the feed (camera_play.py).
- **MediaPipe** - MediaPipe Hands detects the hand; the index-fingertip landmark (id 8) is converted into a movement direction, and a closed fist is detected as a brake (vision/hand_tracker.py).
- **DeepFace** - analyses the dominant facial emotion and maps it to a difficulty multiplier that scales pirate speed (vision/face_analyzer.py).
- **Gymnasium** - wraps the game as an RL environment with reset()/step() and defined action and observation spaces (rl/space_env.py).
- **Stable-Baselines3** - trains a PPO agent (train.py) and runs it autonomously (ai_play.py).

3. Connecting the Camera to the Game

OpenCV captures each frame and flips it horizontally so movement feels like a mirror. The frame is passed to MediaPipe, which returns hand landmarks; the index fingertip's offset from the frame centre is thresholded per-axis (a dead-zone) into a direction vector (dx, dy), each in {-1, 0, 1}, allowing the eight compass directions plus idle. The same frame is passed to DeepFace, whose dominant emotion sets the difficulty multiplier. **No-detection handling:** if no hand is found the ship idles; if no face is found the last stable difficulty is kept. DeepFace runs only once every N frames and reuses its last result, both for performance and to avoid flicker.

4. State / Observation

The observation is a vector of **6 floats, normalized to [0,1]**: [player_x, player_y, ore_x, ore_y, nearest_pirate_x, nearest_pirate_y]. These are the minimum values needed to decide where to go: where I am, where the reward is, and where the closest threat is. Normalization keeps the neural-network inputs in a stable range. **Sufficiency / limitation:** the agent sees only the nearest pirate (not all three) and has no velocity information, so it cannot perfectly anticipate pirate paths - a deliberate simplification that keeps the state small and learnable, and a clear candidate for future extension.

5. Actions

The action space is **Discrete(9)**: idle, the four straight moves, and the four diagonals. The environment maps each integer to a direction vector (the ACTION_VECTORS table) and calls set_velocity(dx, dy); the vector is normalized so diagonal moves are not faster than straight ones. Nine discrete actions give full 2D mobility while keeping the action space discrete, which suits PPO and DQN. (An earlier version used Discrete(5) with no diagonals - comparing the two is a useful experiment.)

6. Reward

The reward signal is: **+1** for collecting ore, **-1** for a pirate collision (terminal), **-0.01** per step to encourage efficiency, plus a small distance-shaping term (**+0.02** when the ship moves closer to the ore, **-0.005** when it moves away). This encourages the agent to reach ore quickly while avoiding pirates. **Reward shaping during development:** the first version used only the sparse +1/-1/-0.01 signal and learned slowly; adding the distance term gave the agent a denser gradient toward the ore and sped up learning - the two variants are compared in the Results section.

7. Policy

The policy is the function (a neural network, `MlpPolicy`) that maps a state to a distribution over the nine actions. During training the agent samples from this distribution; at play time it takes the most likely action (deterministic). PPO updates the policy's weights via policy gradients so that actions which led to higher reward become more probable. The policy depends entirely on the 6-value state described above.

8. Exploration vs. Exploitation

Exploration means trying varied actions to discover which lead to reward; **exploitation** means choosing the actions already known to be good. During training PPO explores by sampling stochastically from the policy (encouraged by an entropy term). At play time we call `predict(obs, deterministic=True)`, which is pure exploitation - always the highest-probability action. Exploration matters most early on: before the agent has ever reached ore or been hit, it must try many actions to gather the experience the policy learns from.

9. Training Process

The agent is trained with **PPO (`MlpPolicy`)** from `Stable-Baselines3`. PPO was chosen as a stable, well-documented on-policy algorithm that works well with discrete actions and small observation vectors. Training runs headless for speed for **[TOTAL_TIMESTEPS, e.g. 500,000]** timesteps and saves `models/space_model.zip`; `ai_play.py` reloads it and acts deterministically. At the start of training the agent moved almost randomly and died quickly; as training progressed it began steering toward ore and surviving longer. The action space was changed from 5 to 9 actions mid-project, which required retraining a fresh model.

10. Results

Performance was measured with the mean episode reward (`ep_rew_mean`) reported by `Stable-Baselines3`, comparing the start of training to the end. Replace the bracketed values below with the numbers from your own training log.

Metric	Before training	After training
Mean episode reward	[~ -1.0]	[your value]
Mean episode length (steps)	[short]	[longer]
Ore collected per episode	[~0]	[your value]

Two reward variants were compared: the sparse signal (+1/-1/-0.01) and the same signal plus distance shaping. The shaped variant reached a positive mean reward in fewer timesteps.

11. Problems Encountered and How They Were Solved

- **Python 3.14 had no wheels** for `pygame/mediapipe/stable-baselines3` - solved by using a Python 3.12 virtual environment.
- **pip and python pointed to different interpreters** (packages installed but not found) - solved by installing with `python -m pip` inside the `venv`.
- **Sprites looked squished** because images were scaled to a square - solved by scaling the longest edge while preserving the native aspect ratio.
- **The pirate sprite looked too small** - solved by decoupling on-screen display size from the collision radius.

- **Audio played too often** (a sound on every episode reset) - solved by playing the launch sound once, lowering the master volume, and adding an M mute toggle.
- **The trained model became incompatible** after changing the action space - solved by deleting and retraining the model.

12. Conclusions

PPO successfully learned to navigate toward ore and avoid the nearest pirate from a compact 6-value state. The hardest parts were the environment setup (Python version and venv issues) and tuning the reward so the agent learned efficiently rather than wandering. Future improvements: include pirate velocities and all pirates in the state, compare PPO with DQN, experiment further with reward shaping, and add levels or moving ore.

13. Division of Work

[Describe how the work was divided among team members, or state that this was an individual project. Example: "Individual project - all components (game, vision, RL environment, training, documentation) implemented by the submitter."]